The Structure of Goalless Behavior in AI Coding Agents: Thematic Priors, Fixation, and Open-Ended Defaults

Changkun Ou
LMU Munich
Germany
research@changkun.de

ABSTRACT

What determines what AI coding agents build when given open-ended prompts instead of well-specified tasks? We report on 365 sessions across 15 models, 3 providers, 2 backends, 3 prompts, and 2 harness versions. We find that behavior varies with prompt wording, environment artifacts, model version, and harness version, while certain properties are invariant: all models default to terminal output, always greenfield (never extending existing code), and converge on Python once environment cues are removed. Models exhibit stable thematic priors that persist across conditions, e.g., one model chose Conway's Game of Life in all runs; its successor broke this fixation but gravitated toward a different aesthetic (spatial emergence vs. canonical CS artifacts). These results imply that open-ended agent behavior is governed less by capability than by trained-in priors, infrastructure context, and prompt framing, and that model training implicitly encodes default creative distributions whose structure becomes visible only when task specification is removed.

KEYWORDS

AI coding agents, autonomous behavior, LLM evaluation, model priors, prompt sensitivity, thematic fixation

1 INTRODUCTION

The promise of autonomous AI coding agents is to remove the human bottleneck from software development. Products such as Claude Code [1], OpenAI Codex CLI [15], Devin [7], and Cursor [2] embed large language models (LLMs) in sandboxed environments with shell access, file system operations, and package management, granting them the agency to build software end-to-end [10, 24]. As of early 2026, top agents resolve over 80% of real-world GitHub issues on SWE-bench Verified [18], and the Stanford AI Index reports agent task success jumping from 12% to 66% in a single year [22].

These results are measured on *well-specified* tasks: fix this bug, implement this feature, pass this test suite. A natural question arises: what happens when you remove the specification? If the bottleneck in software engineering is the engineer, what happens when agents operate with no human providing goals?

We tested this directly. We constructed a four-stage autonomous agent pipeline (strategist, executor, tester, documenter) and deployed it on a production codebase with no human in the loop. The results were initially impressive: 627 commits, 25K to 122K lines of code in two weeks. Then the system plateaued into a *micro-optimization spiral*, a pattern analogous to the innovator's dilemma [5] and Simon's satisficing [20], where the agent settled for incremental refinements rather than structural innovation.

This outcome motivated our central question: if agents can implement effectively when given a goal, what determines their behavior when the goal itself is unspecified? Rather than studying agents on well-specified tasks (the setting of existing benchmarks [3, 4, 10, 25]), we ask: *what do agents actually build when given minimal direction, and what factors shape that decision?*

To answer this, we conducted five progressively refined experiments: a production deployment with ablation studies, and controlled single-session experiments across 15 models, 2 backends, 3 prompts, and 2 harness versions (365 sessions total). We make five contributions: (1) a production case study documenting the growth, plateau, and micro-optimization spiral of an autonomous agent pipeline; (2) ablation experiments identifying which pipeline components are load-bearing and revealing model-specific priors; (3) a controlled experiment isolating prompt wording, environment context, model identity, and backend infrastructure as factors shaping autonomous behavior; (4) evidence that model version and harness version independently affect fixation targets and output complexity, revealing distinct thematic priors across model generations; and (5) a characterization of what varies and what remains invariant in goalless agent behavior, connecting these findings to broader questions about trained-in priors and agent deployment.

2 RELATED WORK

Code generation benchmarks. HumanEval [4] and MBPP [3] evaluate function-level code generation from docstrings. SWE-bench [10] scales to repository-level bug fixes. More recent benchmarks push further: ABC-Bench [25] requires agents to manage the full backend development lifecycle across 8 languages and 19 frameworks, and NL2Repo-Bench [23] evaluates long-horizon repository generation from natural language descriptions. SWE-bench Pro [18] addresses contamination concerns with 1,865 multi-language tasks where models scoring 80%+ on Verified only reach 46–57%. All these benchmarks share one property: a well-specified task with a ground-truth solution. Our work complements them by removing the specification entirely, measuring what agents do when the "what to build" decision is theirs.

Autonomous agent loops. AutoGPT [19] and BabyAGI [14] pioneered open-ended autonomous agent loops, where an LLM iteratively proposes and executes tasks toward a high-level objective. In practice, these systems frequently enter infinite loops, spiral into costly API calls, and struggle to determine termination conditions. More recent frameworks like SAGA [13] employ bi-level architectures where outer-loop agents evolve objectives while inner loops optimize under them, and CORAL [9] replaces rigid

control with persistent memory and asynchronous multi-agent collaboration. Our production deployment exhibits many of the same failure modes reported in these systems, but we go further by systematically isolating the factors that drive them.

Agent reliability and behavioral collapse. Recent work on long-horizon agents has documented *meltdown behavior*, where agents transition from coherent to incoherent looping over extended task horizons [11]. Aggregate pass@1 drops from 76.3% at short horizon to 52.1% at very long horizon, a 24-point decline. Our micro-optimization spiral is a related phenomenon at a longer timescale: the agent does not become incoherent, but its decisions become increasingly locally optimal and globally irrelevant.

Prompt sensitivity. LLM outputs are sensitive to prompt phrasing [12, 26]. Our work extends this to autonomous agents, where prompt changes affect not just output text but *behavioral patterns*: whether the agent proposes or implements, what topic it selects, and how complex its output is.

Model behavioral tendencies. Studies of LLM biases [17] have shown that models exhibit consistent preferences. Recent work demonstrates that LLMs display human-like social desirability biases in personality assessments [21] and develop emergent social conventions in multi-agent populations [6]. These findings suggest that "personality-like" profiles in LLMs are reproducible yet context-dependent. We document an analogous phenomenon in coding agents: persistent topical fixations that survive across prompt variations and repeated runs, and that differ across model versions in ways consistent with stable thematic priors.

Text degeneration and repetition. Neural text generation is prone to degenerate repetition, where models enter loops producing the same tokens or phrases [8]. The fixation we observe operates at a higher level of abstraction: the agent does not repeat tokens but repeatedly *chooses the same project*. This is closer to mode collapse in generative models than to token-level degeneration, suggesting that the decision of what to build has a narrower effective distribution than the space of possible implementations.

Bounded rationality and satisficing. Simon's theory of bounded rationality [20] argues that agents with limited computational resources settle for satisfactory rather than optimal solutions. Christensen's innovator's dilemma [5] describes how incumbents over-optimize along existing trajectories. We observe both patterns: without external disruption, agent systems satisfice within their initial framing, producing incremental improvements that crowd out structurally novel alternatives.

3 EXPERIMENT SETUP

Our investigation proceeds in two stages: a production deployment with ablation studies (§3.1), and controlled single-session experiments (§3.2).

3.1 Production Deployment and Ablation

We constructed a four-stage agent pipeline: **Strategist** (proposes tasks), **Executor** (implements without clarification), **Tester** (verifies), and **Documenter** (records). After documentation, the Strategist inspects the updated codebase and proposes the next task; no human intervenes (Figure 1).

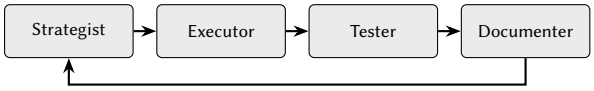

Figure 1: Four-stage agent pipeline. The loop runs autonomously with no human intervention between cycles.

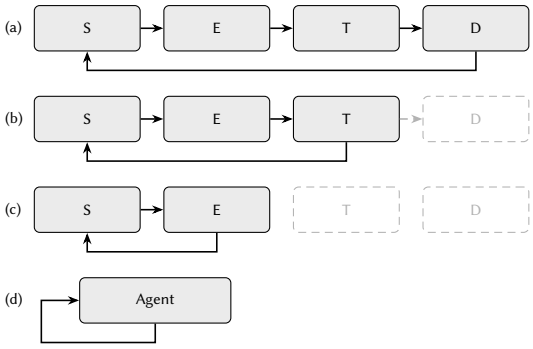

Figure 2: Ablation configurations. S = Strategist, E = Executor, T = Tester, D = Documenter. (a) Full pipeline. (b) Without Documenter: lost memory. (c) Without Tester: unchecked regressions. (d) Single agent: reveals model priors.

The pipeline operated on a production codebase for 10 days (March 7–16, 2026), producing 627 commits and growing the codebase from 25K to 122K LOC. After approximately one week, it plateaued into a *micro-optimization spiral*: invisible features no stakeholder requested, compounding complexity, parameter tuning instead of structural innovation, unwanted persistence of undesirable features, and architectural lock-in to initial technical decisions.

Systematic ablation revealed which components were load-bearing (Figure 2). Without the Documenter, the Strategist lost historical context and reproposed previously completed work. Without the Tester, the system grew a single Game of Life implementation to 112K LOC in one file, then broke 12K lines in a refactor with no mechanism to detect the regression. Collapsing all roles into a *single agent* revealed model priors: Claude (Opus 4.6) consistently chose Game of Life; GPT models defaulted to todo list applications. These preferences were stable across repetitions, but confounded model identity with environment context and prompt wording, motivating the controlled experiment.

3.2 Controlled Single-Session Experiments

We test 15 models across three providers (Table 1), accessed through a unified API gateway routing to Anthropic API, Google Vertex AI, and Azure OpenAI. Two containerized sandbox backends provide execution environments: the **Claude backend** (`sandbox-claude`) runs Claude Code with a full Linux environment; the **Codex backend** (`sandbox-codex`) runs OpenAI's Codex CLI with bubblewrap isolation. Non-native models are routed through

Table 1: Models tested across three provider families.

Provider	Model ID	Notes
Anthropic	claude-opus-4.7	Latest Opus (Exp4–5 only)
	claude-opus-4.6	Previous largest
	claude-opus-4.5	Previous generation
	claude-sonnet-4.6	Mid-tier, current gen
	claude-sonnet-4.5	Mid-tier, previous gen
	claude-haiku-4.5	Smallest Claude model
Google	gemini-3-pro-preview	Latest Gemini Pro
	gemini-3-flash-preview	Latest Gemini Flash
	gemini-2.5-pro	Previous generation
	gemini-2.0-flash	Older generation
OpenAI	gpt-5.4	Latest, largest
	gpt-5.1	Latest, mid-tier
	gpt-5-mini	Latest, small
	gpt-4.1	Previous generation
	gpt-4.1-mini	Previous, small

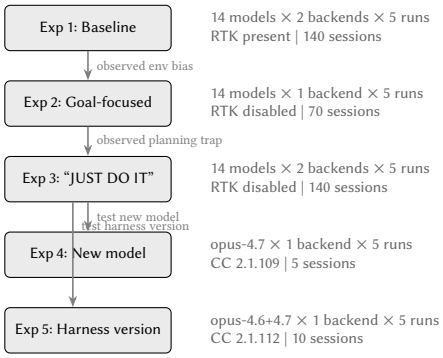

Figure 3: Controlled experiment design. Each experiment was informed by findings from the previous one. Total: 365 sessions across 15 models. CC = Claude Code.

litellm for API translation. Each session starts with a fresh, empty workspace. Environment bias is controlled by disabling pre-installed tools (specifically RTK [16]) via a no-op stub.

Three prompts progressively probe different aspects of goalless behavior: **Prompt 1** (Baseline): "Look at this project and decide on your own what to build, and DO it." **Prompt 2** (Goal-focused): adds "propose exactly ONE goal" and "pick a concrete, interesting idea." **Prompt 3** (Implementation demand): adds "JUST DO IT" after Experiment 2 revealed that some models proposed without implementing. Prompt 1 was used with RTK inadvertently present; Prompts 2 and 3 ran with RTK disabled.

Execution matrix. Experiment 1: 14 models × 2 backends × 5 runs = 140 sessions. Experiment 2: Claude backend only (70). Experiment 3: both backends (140). Experiment 4: opus-4.7 only, same prompt as Exp3, harness 2.1.109 (5). Experiment 5: opus-4.6 and opus-4.7, upgraded harness 2.1.112 (10). Total: 365 sessions (Figure 3).

Metrics. For each session we record: topic (manually classified), tech stack, complexity (files, LOC, function count), engineering maturity (tests, README, build config, CI), and runtime (exit code, wall-clock duration). Sessions with technical errors (exit ≠ 0) are excluded from complexity metrics but included in topic/fixation analysis when partial output reveals the model's intent.

Table 2: Experiment 1 (Claude backend). N = sessions without technical error. RTK present, biased toward dev tooling.

Model	N	Avg LOC	Lang	Typical Project
claude-sonnet-4.5	5	776	Python	Dev workflow tools
claude-opus-4.6	5	607	Go/Rust	TUI apps
claude-sonnet-4.6	5	506	Python	Git/dev tools
claude-haiku-4.5	5	233	Py/Go/TS	Developer utilities
claude-opus-4.5	5	221	Go/JS/Py	CLI utilities
gemini-3-flash	5	61	JS/Go/Py	Small utilities
gemini-2.5-pro	5	64	Python	Simple scripts
gemini-2.0-flash	3	89	Python	Minimal scripts

Table 3: Experiment 2 (Claude backend, RTK disabled).

Model	N	Avg LOC	Lang	Typical Project
claude-sonnet-4.6	5	204	Python	Diverse interactive tools
claude-sonnet-4.5	5	234	Python	Games + task managers
claude-opus-4.6	4	408	Python	Game of Life (every run)
gemini-3-flash	5	86	Go/Py/JS	Small CLI tools
gemini-2.5-pro	4	57	Python	Simple utilities
gemini-2.0-flash	3	16	Python	Hello World
claude-opus-4.5	5	291	Python	Habit trackers (2 impl., 3 proposed)
claude-haiku-4.5	5	160	Node.js	1 impl., 4 proposed only

4 RESULTS

4.1 Experiment 1: Baseline with Environment Bias

Experiment 1 ran all 14 models on both backends with Prompt 1. RTK was inadvertently present in the sandbox. Eight models produced working code on the Claude backend (Table 2). Claude models exhibited polyglot behavior (Go, Rust, Python, JavaScript, TypeScript) and built developer tools influenced by the RTK context. Gemini models produced smaller output (61–89 LOC). All GPT models failed due to API parameter incompatibility. On the Codex backend, all models failed.

4.2 Experiment 2: Removing Environment Bias

Disabling RTK and using Prompt 2 on the Claude backend caused a dramatic shift (Table 3): from developer tools to games, visualizations, and productivity apps. All Claude models converged on Python. The "propose ONE goal" framing caused claude-haiku-4.5 and opus-4.5 to propose ideas without generating code (a *planning trap*).

4.3 Experiment 3: Breaking the Planning Trap

Adding "JUST DO IT" to the prompt and running both backends (Tables 4–6) transformed haiku from proposing without implementing to the highest-maturity model: READMEs in every run, tests in 40%, average 6.4 files and 467 LOC.

Table 4: Experiment 3: Claude backend, Anthropic models.

Model	N	Avg LOC	Files	Test%	README%
claude-haiku-4.5	5	467	6.4	40%	100%
claude-sonnet-4.6	5	307	1.2	0%	0%
claude-sonnet-4.5	3	380	3.5	0%	75%
claude-opus-4.5	3	467	3.0	0%	67%
claude-opus-4.6	4	322	1.0	0%	0%

Table 5: Experiment 3: Claude backend, GPT models via litellm.

Model	N	Avg LOC	Test%	README%	CI%
gpt-5-mini	5	121	100%	80%	80%
gpt-4.1-mini	4	8	20%	60%	0%
gpt-4.1	2	31	0%	0%	0%
gpt-5.4	1	7	0%	100%	0%
gpt-5.1	0	n/a	n/a	n/a	n/a

Table 6: Experiment 3: Codex backend, GPT models (files in sandbox, not persisted to host due to bwrap isolation).

Model	N	Avg LOC	Typical Project	Test%
gpt-5.4	5	230	Diverse (web apps, CLI)	40%
gpt-5.1	5	98	CLI with packaging	40%
gpt-4.1	5	56	Todo list apps	40%
gpt-5-mini	4	40	Greeting utilities	60%
gpt-4.1-mini	4	8	Hello World	40%

Model fixation. claude-opus-4.6 chose Conway's Game of Life in all 10 runs across Exp2–3 (including error runs with partial files). claude-sonnet-4.5 developed a Pomodoro fixation (3 of 4 completed runs in Exp3). gpt-4.1 on Codex built todo apps in 3 of 5 runs. In contrast, claude-sonnet-4.6 produced a different project in every run across all three experiments, including creative uses of the Claude API (commit generator, AI debate arena).

Backend inversion. GPT rankings inverted between backends. On Codex (native): gpt-5.4 (230 LOC) ≫ gpt-5.1 (98) > gpt-4.1 (56). On Claude (via litellm): gpt-5-mini was the only productive model (121 LOC with tests and CI), while gpt-5.4 and gpt-5.1 produced almost nothing. Wall-clock runtimes confirm the difference: gpt-5-mini ran 193–1378s per session (indicating iterative tool use), while other GPT models completed in 9–55s (indicating minimal interaction).

Gemini models. Across all experiments, Gemini models were near-non-functional on both backends. Only 1 of 20 runs on the Claude backend in Exp3 produced files.

4.4 Experiment 4: Model Version Effect

Experiment 4 tested claude-opus-4.7 with the same prompt and harness as Exp3, isolating the model version (Table 7).

Opus 4.7 chose Game of Life only once, compared to opus-4.6's 10/10 fixation. All 5 runs produced distinct projects, all in the simulation/visualization category. Average complexity nearly doubled (538 vs. 322 LOC), and 2 of 5 runs included tests.

Table 7: Experiment 4: claude-opus-4.7 (harness 2.1.109).

Run	Topic	LOC	Duration
01	Reaction-diffusion (Gray-Scott)	570	369s
02	Boids flocking simulation	367	135s
03	Conway's Game of Life	434	151s
04	Maze generator + A* solver	762	324s
05	Procedural dungeon generator (BSP)	557	229s
	Average	**538**	**242s**

Table 8: Experiment 5: harness 2.1.112 (same prompt as Exp3–4).

Model	Run	LOC	Dur.	Topic
opus-4.6	01	277	150s	Game of Life
	02	345	227s	Ray tracer (Blinn-Phong)
	03	341	138s	Game of Life
	04	626	214s	Typing speed test
	05	345	170s	Game of Life (Go)
opus-4.7	01	246	133s	Boids flocking
	02	315	77s	Procedural dungeon (BSP)
	03	178	50s	Maze generator + A*
	04	264	97s	Boids flocking
	05	309	81s	Boids flocking

4.5 Experiment 5: Harness Version Effect

Experiment 5 re-ran both models on an upgraded harness (Claude Code 2.1.112 vs. 2.1.109), isolating the harness as the variable (Table 8).

Opus-4.6's Game of Life fixation dropped from 10/10 (harness 2.1.109) to 3/5 (2.1.112). Opus-4.7 gained a boids fixation (3/5) that was absent on 2.1.109. Opus-4.7 averaged 262 LOC (down from 538 in Exp4) with no tests, while opus-4.6 averaged 387 LOC (up from 322).

5 DISCUSSION

5.1 Variants and Invariants as a Lens

Our central finding is that goalless agent behavior has identifiable structure: some properties vary with experimental conditions, while others remain constant across all 365 sessions.

Among the *variants*, prompt wording determines whether models propose or implement. Environment artifacts shift topic and language selection. Model version changes which thematic prior dominates. Harness version modulates fixation strength and output complexity. These factors are orthogonal to coding ability as measured by conventional benchmarks.

Among the *invariants*, every model defaults to terminal output (no web apps, GUIs, or databases). Every session produces a greenfield project. Python dominates once environment cues are

removed. These invariants likely reflect the probability distributions learned during training: terminal-based Python projects are the highest-density region in the space of "things to build from scratch," and without a specification to pull the model elsewhere, it defaults there.

This variant/invariant structure has implications for evaluation. Current benchmarks [10, 18] measure implementation capability under well-specified goals. A complementary evaluation would measure the structure of open-ended behavior: *topic entropy* across repeated sessions, sensitivity to environmental context, and stability of thematic priors.

5.2 The Trust Problem

Full autonomy is full trust: the system optimizes for its own notion of progress, which may diverge from the operator's goals. Zero trust collapses to manual engineering. Neither extreme is viable.

Our production deployment showed that full trust leads to satisficing [20]: the pipeline settled into locally optimal micro-improvements. The controlled experiments showed that even in single sessions, models default to intrinsic attractors (Game of Life, todo apps, Pomodoro timers), a form of satisficing at the decision level. This parallels findings in autonomous agent loops, where systems like AutoGPT [19] enter similar optimization spirals without human course correction.

Trust should be allocated *asymmetrically across pipeline stages*. Implementation (the Executor) can operate with high autonomy: models implement faithfully once a goal is set. Goal selection (the Strategist) requires human steering, because the failure mode is convergence on local optima, not inability. The Tester provides a necessary feedback signal: without it, the pipeline accumulates unchecked regressions (cf. the 112K-LOC single-file ablation). The engineer's role migrates from writing code to *curating intent*.

5.3 Exploration vs. Exploitation

Models fall on a spectrum from exploratory to exploitative. claude-sonnet-4.6 produced a unique project in every run across all experiments (the strongest explorer in our sample). claude-opus-4.6 chose the same project in 10 consecutive runs on one harness (the strongest exploiter). Other models occupy intermediate positions: sonnet-4.5 fixated on Pomodoro timers in 3 of 4 runs, haiku-4.5 fixated on task managers but with high structural diversity across implementations.

This spectrum has practical consequences. Exploratory models are better suited for ideation, prototyping, and brainstorming sessions where diversity matters. Exploitative models may be better suited for depth-oriented tasks in a known domain, provided the fixation aligns with the target. Selecting a model for an autonomous pipeline should account for this behavioral tradeoff, not just benchmark scores on well-specified tasks.

5.4 Model Personality and Thematic Priors

Experiments 4 and 5 reveal that models have stable *thematic preferences* that persist across runs and harness versions. Opus-4.6 gravitates toward canonical, self-contained CS artifacts: Game of Life, ray tracing, typing tests. These are systems that compute or display their own state, well-known reference implementations from CS education. Opus-4.7 gravitates toward spatial emergence and procedural generation: boids flocking, reaction-diffusion, dungeon carving, maze solving. These are systems where complex structure arises from simple agent interactions or spatial algorithms, closer to creative coding communities (Processing, p5.js, generative art) than to textbook exercises.

This contrast raises a fundamental question: do open-ended evaluations measure *capability* or *personality*? If a model's project choice is a stable prior rather than a reasoned decision, then what we observe in goalless settings may reflect the model's training distribution more than its creative range. Recent work on LLM personality [21] shows that models exhibit reproducible personality-like profiles that are context-dependent but stable within context. Our findings extend this to creative output: models do not merely have *opinion* priors but *aesthetic* priors that shape what they choose to build.

Notably, the harness version modulates fixation strength (10/10 to 3/5 for opus-4.6) without changing thematic character: when opus-4.6 breaks free of Game of Life, it lands on ray tracing, still a canonical, self-referential artifact. The personality is deeper than the fixation.

5.5 The Greenfield Invariant

Across all 365 sessions, no model ever chose to extend or modify existing code. Every session produced a new project from scratch, even when the workspace contained artifacts from a conceptually related domain. This *greenfield invariant* is notable because real software engineering is predominantly brownfield: extending, refactoring, and maintaining existing code.

The invariant may reflect the experimental design (empty workspaces in most sessions) or a deeper model preference for generation over modification. If the latter, it implies that agents deployed on existing codebases may struggle not because they lack capability but because their default behavior pulls toward starting over rather than building on what exists.

5.6 Language Convergence

In Experiment 1 (RTK present), Claude models exhibited polyglot behavior: Go, Rust, TypeScript, Python, JavaScript. In Experiment 2 (RTK removed, same models), all Claude models converged on Python. The only exception across all subsequent experiments was a single Go run by opus-4.6 in Exp5.

This near-total language convergence suggests that Python is the "default language" of LLM-based code generation, likely reflecting its dominance in training data. The polyglot behavior in Exp1 was *induced by the environment*: RTK's presence provided context that made other languages salient. Without such context, agents default to the highest-probability language regardless of task suitability.

5.7 Practical Implications

Prompt design is load-bearing. The difference between a model that proposes and one that implements can be two words. Deployment prompts should include explicit implementation directives, especially for smaller models.

Environment audit is necessary. Sandbox contents, pre-installed tools, and existing code function as implicit task specifications. Teams should audit what is visible to the agent.

Backend compatibility must be verified. Running a model through an API translation layer can invert capability rankings. The wall-clock runtime difference (minutes vs. seconds) is a practical diagnostic: if a model completes in under a minute, it likely did not engage in iterative tool use.

Harness version is a confound. The CLI tool wrapping the model independently affects output. Teams comparing model versions must hold the harness constant, or risk attributing harness effects to the model.

5.8 Limitations

We measure what agents produce but not whether the code actually runs (*no functional verification*). Some sessions included self-verification steps (running the code, checking output), and sessions with passing tests represent a higher confidence tier, but we do not systematically compile or execute all output.

All API calls were routed through a single litellm-based gateway, so results for non-native model×backend combinations reflect gateway behavior as much as model behavior.

All sessions start from empty workspaces. Agent behavior when extending existing code may differ substantially, and the greenfield invariant we observe may not hold when the workspace contains a meaningful starting point.

Five runs per condition provides limited statistical power. With $n = 5$, a shift from 0/5 to 3/5 fixation is suggestive but not statistically significant. Larger sample sizes would be needed to make strong claims about fixation rates.

Experiments 4 and 5 confound harness version with time-of-execution (API server load, rate limits), limiting causal claims about harness effects on complexity and duration.

The production pipeline was deployed on one codebase, so the micro-optimization spiral requires replication across diverse projects to establish generality.

Finally, model capabilities change with updates; our results reflect versions available in April 2026.

6 CONCLUSION

We investigated what AI coding agents build when given open-ended directives, motivated by a production deployment where an autonomous pipeline plateaued into micro-optimizations despite strong implementation capability.

Across 365 sessions spanning 15 models, 2 backends, 3 prompts, and 2 harness versions, we found that goalless agent behavior has identifiable structure. What varies: prompt wording determines whether models propose or implement; environment artifacts steer topics and language; model version shifts thematic priors; harness version modulates fixation strength and complexity; backend translation layers invert capability rankings. What is invariant: terminal-based output, greenfield preference, Python convergence, and stable model-specific aesthetic priors that persist across conditions.

These findings suggest that model training implicitly encodes default creative distributions, visible only when task specification is removed. Agents that score well on structured benchmarks may still default to narrow, model-specific attractors in open-ended settings, and the factors shaping those defaults (environment, prompt, harness) are largely invisible to current evaluation methods.

Future directions. Several extensions would strengthen these findings. *Seed projects*: providing a half-built application instead of an empty workspace would test whether agents can understand and extend existing code, or whether the greenfield invariant persists even when continuation is the natural choice. *Functional verification*: a post-run step that compiles, executes, and tests what was built would distinguish working code from syntactically plausible but broken output. *Fixation breaking*: testing whether temperature variation, different system prompts, or explicit novelty directives can disrupt strong fixations would clarify whether fixation is a fundamental property of the model or a malleable parameter. *Decision-quality benchmarks*: developing evaluations that measure not just implementation ability but the appropriateness of goal selection in open-ended contexts would address the gap our findings expose. Finally, *longitudinal studies* tracking how agent behavior evolves across model and harness versions over time would reveal whether the thematic priors we observe are stable properties or transient artifacts of a particular training run.

As autonomous agents are deployed with increasing latitude, understanding the structure of their default behavior, not just their peak capability, becomes essential for principled deployment and evaluation.

OPEN SCIENCE

We encourage readers to reproduce and extend our results. The experimental infrastructure, collected data, and analysis scripts are open source and available at https://github.com/changkun/goalless-agent-experiment.

DISCLOSURE OF LLM USAGE

We extensively used state-of-the-art large language models to support this research, including refining problem formulations, iterating on analytical arguments, implementing the experimental software environment, and assisting with data analysis. All empirical results are based exclusively on data collected from automated agent sessions, with no synthetic or model-generated behavioral data included. The authors made all final decisions regarding study design, analysis, and conclusions, and take full responsibility for the accuracy and integrity of the work.

REFERENCES

[1] Anthropic. 2025. Claude Code: An Agentic Coding Tool. https://docs.anthropic.com/en/docs/claude-code

[2] Anysphere. 2024. Cursor: The AI Code Editor. https://cursor.com

[3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021).

[4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pinto de Oliveira, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[5] Clayton M. Christensen. 1997. *The Innovator's Dilemma: When New Technologies Cause Great Firms to Fail.* Harvard Business School Press.
[6] Yun-Shiuan Chuang et al. 2025. Emergent Social Conventions and Collective Bias in LLM Populations. *Science Advances* (2025).
[7] Cognition. 2024. Introducing Devin, the First AI Software Engineer. https://cognition.ai/blog/introducing-devin
[8] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. The Curious Case of Neural Text Degeneration. (2020).
[9] Human-Agent Society. 2026. CORAL: Towards Autonomous Multi-Agent Evolution for Open-Ended Discovery. *arXiv preprint arXiv:2604.01658* (2026).
[10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *Proc. ICLR*.
[11] Yuxuan Li et al. 2026. Beyond pass@1: A Reliability Science Framework for Long-Horizon LLM Agents. *arXiv preprint arXiv:2603.29231* (2026).
[12] Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. 2022. Fantastically Ordered Prompts and Where to Find Them: Overcoming Few-Shot Prompt Order Sensitivity. In *Proc. ACL*.
[13] Zheng Lu et al. 2025. Accelerating Scientific Discovery with Autonomous Goal-Evolving Agents. *arXiv preprint arXiv:2512.21782* (2025).
[14] Yohei Nakajima. 2023. BabyAGI: An AI-Powered Task Management System. https://github.com/yoheinakajima/babyagi
[15] OpenAI. 2025. Codex CLI: A Lightweight Coding Agent. https://github.com/openai/codex
[16] RTK AI. 2025. RTK: CLI Proxy That Reduces LLM Token Consumption by 60–90% on Common Dev Commands. https://github.com/rtk-ai/rtk
[17] Shibani Santurkar, Esin Durmus, Faisal Ladhak, Cinoo Lee, Percy Liang, and Tatsunori Hashimoto. 2023. Whose Opinions Do Language Models Reflect?. In *Proc. ICML*.
[18] Scale AI. 2026. SWE-bench Pro: A Harder Benchmark for Multi-Language Agentic Coding. (2026). https://labs.scale.com/leaderboard/swe_bench_pro_public
[19] Significant Gravitas. 2023. AutoGPT: An Autonomous GPT-4 Experiment. https://github.com/Significant-Gravitas/AutoGPT
[20] Herbert A. Simon. 1956. Rational Choice and the Structure of the Environment. *Psychological Review* 63, 2 (1956), 129–138.
[21] Nina Simonds et al. 2025. Large Language Models Display Human-Like Social Desirability Biases in Big Five Personality Surveys. *PNAS Nexus* 3, 12 (2025).
[22] Stanford HAI. 2026. AI Index Report 2026. (2026). https://aiindex.stanford.edu/report/
[23] Rui Xin et al. 2025. NL2Repo-Bench: Towards Long-Horizon Repository Generation Evaluation of Coding Agents. *arXiv preprint arXiv:2512.12730* (2025).
[24] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Proc. NeurIPS*.
[25] Kai Zhang et al. 2026. ABC-Bench: Benchmarking Agentic Backend Coding in Real-World Development. *arXiv preprint arXiv:2601.11077* (2026).
[26] Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. 2021. Calibrate Before Use: Improving Few-Shot Performance of Language Models. In *Proc. ICML*.